

INVESTIGAÇÃO EXPERIMENTAL DE ESTRUTURAS DE DADOS E ALGORITMOS

Linguagem: Python

Objetivo: investigar experimentalmente arrays, matrizes, ordenação, busca, índices, loops e complexidade computacional, medindo e comparando operações realizadas pelos algoritmos.

PARTE 1 — PESQUISA: BUBBLE SORT E QUICK SORT

1.1 Bubble Sort

O Bubble Sort percorre o array comparando elementos vizinhos. Quando estão fora de ordem, eles são trocados. O processo é repetido até que o array esteja ordenado.

Complexidade: melhor caso $O(n)$ com otimização e array já ordenado; caso médio $O(n^2)$; pior caso $O(n^2)$; memória adicional $O(1)$.

Vantagens: simples, didático e usa pouca memória. **Limitações:** muitas operações em arrays grandes. **Adequado:** conjuntos pequenos e estudos. **Não recomendado:** grandes conjuntos.

1.2 Quick Sort

O Quick Sort usa divisão e conquista. Escolhe um pivô, particiona o array em valores menores e maiores e aplica o processo recursivamente às partes.

Complexidade: melhor $O(n \log n)$, média $O(n \log n)$, pior $O(n^2)$. Memória geralmente $O(\log n)$ pela recursão, podendo chegar a $O(n)$ no pior caso.

Vantagens: normalmente muito eficiente para grandes conjuntos. **Limitações:** depende da escolha do pivô e pode atingir $O(n^2)$.

1.3 Comparação

Característica	Bubble Sort	Quick Sort
Princípio	Compara e troca vizinhos	Divide usando pivô
Melhor caso	$O(n)$	$O(n \log n)$
Caso médio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$
Memória adicional	$O(1)$	$O(\log n)$ média
Vantagem	Simplicidade	Eficiência média
Limitação	Crescimento quadrático	Pior caso $O(n^2)$
Uso	Arrays pequenos	Arrays médios/grandes

PARTE 2 — EXPERIMENTO DE ORDENAÇÃO

O mesmo array original é copiado para os dois algoritmos. Foram usados 10, 20 e 1.000 elementos, contando comparações, trocas do Bubble Sort e movimentações do Quick Sort.

```
import random
```

```
def bubble_sort(a):
    comparacoes = trocas = 0
    n = len(a)
    for i in range(n-1):
        trocou = False
        for j in range(n-1-i):
            comparacoes += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                trocas += 1
```

```

        trocou = True
    if not trocou:
        break
    return comparacoes, trocas

def quick_sort(a):
    comparacoes = movimentacoes = 0
    def ordenar(inicio, fim):
        nonlocal comparacoes, movimentacoes
        if inicio >= fim:
            return
        pivo = a[fim]
        i = inicio
        for j in range(inicio, fim):
            comparacoes += 1
            if a[j] <= pivo:
                if i != j:
                    a[i], a[j] = a[j], a[i]
                    movimentacoes += 1
                i += 1
        if i != fim:
            a[i], a[fim] = a[fim], a[i]
            movimentacoes += 1
        ordenar(inicio, i-1)
        ordenar(i+1, fim)
    ordenar(0, len(a)-1)
    return comparacoes, movimentacoes

def gerar_array(n):
    random.seed(12345+n)
    return [random.randint(0,9999) for _ in range(n)]

for n in [10,20,1000]:
    original = gerar_array(n)
    cb,tb = bubble_sort(original.copy())
    cq,mq = quick_sort(original.copy())
    print(n, cb, tb, cq, mq)

```

Tamanho	Bubble Comparações	Bubble Trocas	Quick Comparações	Quick Movimentações
10	44	19	37	7
20	175	112	63	42
1.000	498.275	231.438	10.816	5.416

Os valores dependem dos dados e da escolha do pivô. O código fixa a semente aleatória para tornar o experimento reproduzível.

- a) Para 10 elementos, o Quick Sort realizou menos operações.
- b) Para 20 elementos, o Quick Sort continuou realizando menos operações.
- c) Com 1.000 elementos, a diferença tornou-se muito grande, com o Bubble Sort realizando centenas de milhares de comparações.
- d) O Bubble Sort apresentou maior crescimento.
- e) Sim. Os resultados são coerentes com $O(n^2)$ para Bubble Sort e $O(n \log n)$ no caso médio do Quick Sort.
- f) Bubble Sort: arrays pequenos, didática ou quando simplicidade importa.
- g) Quick Sort: conjuntos maiores em que eficiência é importante.

PARTE 3 — INVESTIGAÇÃO DE BUSCA EM MATRIZES

```

def busca_matriz(matriz, valor):
    comparacoes = 0
    for i in range(len(matriz)):
        for j in range(len(matriz[i])):
            comparacoes += 1
            if matriz[i][j] == valor:
                return True, i, j, comparacoes
    return False, -1, -1, comparacoes

def criar_matriz(linhas, colunas):
    matriz = []
    numero = 1
    for i in range(linhas):
        linha = []
        for j in range(colunas):
            linha.append(numero)
            numero += 1
        matriz.append(linha)
    return matriz

```

```
def testar(matriz, fim):
    print(busca_matriz(matriz, 1))
    print(busca_matriz(matriz, fim))
    print(busca_matriz(matriz, -1))

testar(criar_matriz(2,2),4)
testar(criar_matriz(10,10),100)
testar(criar_matriz(100,100),10000)
```

Matriz	Elementos	Início	Final	Inexistente
2 × 2	4	1	4	4
10 × 10	100	1	100	100
100 × 100	10.000	1	10.000	10.000

- a) O início exige menos operações porque a busca começa pela primeira posição.
- b) Se o valor não existe, todas as posições precisam ser verificadas.
- c) O pior caso ocorre quando o valor está na última posição ou não existe.
- d) Matrizes maiores possuem mais posições e podem exigir mais comparações.
- e) Para m linhas e n colunas, a complexidade é $O(m \times n)$.

PARTE 4 — HANDS ON 1: INVESTIGAÇÃO DO ARRAY

```
temperatura = []

for i in range(10):
    temperatura.append(float(input(f"Digite a temperatura {i}: ")))

for i in range(10):
    print(f"Índice {i}: {temperatura[i]}")

soma = 0
for i in range(10):
    soma += temperatura[i]

media = soma / 10

maior = menor = temperatura[0]
indice_maior = indice_menor = 0

for i in range(1,10):
    if temperatura[i] > maior:
        maior = temperatura[i]
        indice_maior = i
    if temperatura[i] < menor:
        menor = temperatura[i]
        indice_menor = i

acima_media = 0
for i in range(10):
    if temperatura[i] > media:
        acima_media += 1

print("Média:", media)
print("Maior:", maior, "Índice:", indice_maior)
print("Menor:", menor, "Índice:", indice_menor)
print("Acima da média:", acima_media)
A complexidade é  $O(n)$ . Os percursos são lineares e realizados separadamente; somar vários percursos  $O(n)$  continua sendo  $O(n)$ .
```

PARTE 5 — HANDS ON 2: MONITORAMENTO DE SENSORES

```
sensores = []

for i in range(5):
    sensor = []
    for j in range(24):
        sensor.append(float(input(f"Sensor {i}, hora {j}: ")))
    sensores.append(sensor)

for i in range(5):
    soma = 0
    for j in range(24):
        soma += sensores[i][j]
    print(f"Média sensor {i}: {soma/24:.2f} °C")

maior = sensores[0][0]
sensor_maior = horario_maior = 0
for i in range(5):
```

```

    for j in range(24):
        if sensores[i][j] > maior:
            maior = sensores[i][j]
            sensor_maior = i
            horario_maior = j

soma_geral = 0
for i in range(5):
    for j in range(24):
        soma_geral += sensores[i][j]

media_geral = soma_geral / 120
limite = float(input("Limite: "))

acima = 0
for i in range(5):
    for j in range(24):
        if sensores[i][j] > limite:
            acima += 1

print("Maior:", maior)
print("Sensor:", sensor_maior)
print("Horário:", horario_maior)
print("Média geral:", media_geral)
print("Acima do limite:", acima)

```

São necessários loops aninhados porque a matriz possui duas dimensões. i representa o sensor e j representa o horário. Existem $5 \times 24 = 120$ posições. Um percurso completo possui complexidade $O(m \times n)$.

PARTE 6 — ANÁLISE E CONCLUSÃO

1. Sim. O aumento do tamanho da estrutura influencia a quantidade de operações, embora o crescimento dependa da complexidade do algoritmo.
2. Não. Bubble Sort possui crescimento médio $O(n^2)$, enquanto Quick Sort possui $O(n \log n)$ no caso médio.
3. O resultado final não mostra o esforço necessário para produzi-lo. Dois algoritmos podem ordenar o mesmo array, mas realizar quantidades muito diferentes de operações.

CONCLUSÃO

Os experimentos demonstraram a relação entre tamanho da entrada, número de operações e eficiência. Bubble Sort e Quick Sort produzem o mesmo resultado, mas apresentam comportamentos diferentes quando a entrada aumenta.

O Bubble Sort apresentou crescimento muito maior devido à complexidade média $O(n^2)$, enquanto o Quick Sort apresentou comportamento mais eficiente no experimento, coerente com $O(n \log n)$ no caso médio.

Na busca sequencial, a posição do elemento influencia diretamente o número de comparações. Um elemento no início pode exigir uma comparação, enquanto um elemento no final ou inexistente exige a análise de todas as posições.

Os exercícios de arrays e matrizes demonstraram a importância dos índices e loops. Em uma matriz, a quantidade de posições é determinada por linhas $\times$ colunas.

Conclui-se que avaliar somente se um algoritmo funciona não é suficiente. É necessário analisar sua eficiência e complexidade, principalmente quando o tamanho dos dados aumenta.